

**OPERATING SYSTEM: TLB VISUALIZER PRO:
TRANSLATION LOOKASIDE BUFFER**

**A COURSE BASED PROJECT REPORT SUBMITTED IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE OF**

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS

BY

Abhiram	24071A3202
Anish	24071A3203
Lokesh	24071A3205
Chatradhara	24071A3214
Hanesh	24071A3224

UNDER THE GUIDANCE OF

Indurthi Ravindra kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VALLURUPALLI NAGESWARARAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING&TECHNOLOGY
VignanaJyothiNagar,Bachupally,Hyderabad500118,Telangana India

MAY 2026



VNRVJIET

**VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY**

AUTONOMOUS INSTITUTION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Course Based Project titled “**TLB VISUALIZER PRO: TRANSLATION LOOKASIDE BUFFER**” is a record of bona fide work submitted by:

- | | |
|---------------|------------|
| • Abhiram | 24071A3202 |
| • Anish | 24071A3203 |
| • Lokesh | 24071A3205 |
| • Chatradhara | 24071A3214 |
| • Hanes | 24071A3224 |

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **COMPUTER SCIENCE AND BUSINESS SYSTEMS** of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**. The work has been carried out under our guidance and supervision at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

Signature of Supervisor
Indurthi Ravindra kumar
Assistant Professor
Computer science and Engineering
VNR VJIET, Hyderabad

Signature of HOD
r. V Baby
Associate Professor & HOD
Computer science and Engineering
VNR VJIET, Hyderabad

APPROVAL CERTIFICATE

Internship evaluation for the Course Based Project titled “**TLB VISUALIZER PRO: TRANSLATION LOOKASIDE BUFFER**”, being submitted by

Mr. Abhiram, Regd. No. **24071A3202**,

Mr. Anish, Regd. No. **24071A3203**,

Mr. Lokesh, Regd. No. **24071A3205**,

Mr. Chatradhara, Regd. No. **24071A3214**,

Mr. Hanes, Regd. No. **24071A3224**,

has been conducted on _____ and the work is approved for the award of the degree of **BACHELOR OF TECHNOLOGY (B.Tech.)** in **COMPUTER SCIENCE AND BUSINESS SYSTEMS**, under VNRVJIET B.Tech. R22 Academic Regulations.

INTERNAL EXAMINER

EXTERNAL EXAMINER



VNRVJIET

**VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY**

AUTONOMOUS INSTITUTION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the undersigned, declare that the Course Based Project titled **“TLB VISUALIZER PRO: TRANSLATION LOOKASIDE BUFFER”** has been carried out by us in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **COMPUTER SCIENCE AND BUSINESS SYSTEMS** of **Jawaharlal Nehru Technological University Hyderabad (JNTUH)**, under **VNRVJIET B.Tech. R22 Academic Regulations**, at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

We further declare that this work is original, has not been submitted previously for any degree or similar title, and complies with the Institute’s plagiarism and academic ethics guidelines, including AI-generated content. The contributions of all team members are clearly indicated in the report.

PLACE: HYDERABAD

DATE: _____

Signature of the Student(s)

Abhiram	24071A3202
Anish	24071A3203
Lokesh	24071A3205
Chatradhara	24071A3214
Hanesh	24071A3224

ABSTRACT

Memory management is a critical component of modern operating systems, relying heavily on paging to provide isolated, contiguous virtual address spaces to processes. However, mapping logical addresses to physical frames introduces significant latency due to the necessity of multiple memory accesses. The Translation Lookaside Buffer (TLB) acts as a specialized, high-speed associative cache that stores recent translations, drastically reducing Effective Memory Access Time (EMAT). Understanding the internal mechanics of the TLB, including address splitting, hit/miss resolution, and page replacement policies (LRU, FIFO, Random), is conceptually challenging for students. This project, "TLB Visualizer Pro," introduces a highly interactive, web-based simulation environment that dynamically visualizes the address translation pipeline. Developed using vanilla JavaScript, HTML5, and SVG-based animations, the simulator allows users to configure TLB size, physical frames, page bit-sizes, and replacement policies in real-time. By inputting custom page reference strings, users can observe step-by-step animations of the CPU generating a logical address, the associative TLB search, Page Table lookups upon misses, and the subsequent loading of frames into Physical RAM. The system provides real-time statistical tracking of hit/miss ratios, EMAT calculations, and the internal state of the TLB, bridging the gap between theoretical OS concepts and practical, observable metrics.

Keywords: Translation Lookaside Buffer, Memory Management, Operating Systems, Interactive Simulation, Page Replacement Algorithms, Effective Memory Access Time.

TABLE OF CONTENTS

CERTIFICATE	i
APPROVAL CERTIFICATE	ii
DECLARATION	iii
ABSTRACT.....	iv
TABLE OF CONTENTS.....	v
LIST OF FIGURES	vii
LIST OF TABLES.....	viii
ACKNOWLEDGEMENT	ix
CHAPTER 1	1
1.1 Background of the Problem	1
1.2 Need and Relevance.....	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope of Work	3
1.6 Organization of the Report.....	3
CHAPTER 2	4
2.1 Existing System	4
2.1.1 Limitations of Static Pedagogical Methodologies	4
2.1.2 Complexity of Manual Performance Analysis.....	4
2.1.3 Gap between Theory and Implementation.....	5
2.1.4 Accessibility and Setup Constraints.....	5
2.2 Proposed System.....	6
2.2.1 Dynamic SVG-Based Visualization Pipeline	6
2.2.2 Real-Time Parameter Configurability.....	6
2.2.3 Automated Performance Analytics Dashboard.....	7
2.2.4 Educational Features: Step-Through and Flush.....	7
2.2.5 Modern, Accessible Architecture.....	7
2.3 Comparison of Systems	8

CHAPTER 3	9
3.1 Objectives	9
3.2 Selection of Approach / Methodology	10
3.3 System Requirements (Hardware, Software, Functional, Non-Functional)	11
3.3.1 Hardware Requirements.....	11
3.3.2 Software Requirements	11
3.3.3 Functional Requirements	12
3.3.4 Non-Functional Requirements	12
3.4 Architecture Diagram & Explanation	13
CHAPTER 4	15
4.1 Implementation Plan	15
4.2 Modules.....	15
4.3 Module-wise Description and Webpages	16
4.4 Implementation (Include Sample Code)	17
CHAPTER 5	21
5.1 Testing / Evaluation Methodology.....	21
5.2 Results Obtained	21
5.3 Analysis and Interpretation of Results	23
5.3.1 Mathematical Validation of Simulation Results	23
5.3.2 Qualitative Interpretation of Locality	24
5.4 Comparative Performance Metrics and Visualization	24
CHAPTER 6	26
6.1 Conclusions.....	26
6.2 Limitations of the Work.....	26
6.3 Future Scope	26

LIST OF FIGURES

Fig. No.	Title	Page No.
3.1	Architecture Diagram	13
5.1	Initial Configuration and Setup	21
5.2	Visualizing a TLB Miss & Page Fault	22
5.3	Visualizing a TLB Hit	22
5.4	TLB Flush Operation (Context Switch)	23
5.5	Impact of TLB Capacity on Hit Ratio	25
5.6	Reduction in EMAT as TLB Size Increases	25

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Systems	8
5.1	Results Table	24

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and appreciation to all individuals and organizations who have supported us during the execution of our Course Based Project titled **“TLB VISUALIZER PRO: TRANSLATION LOOKASIDE BUFFER”**.

- First and foremost, we are profoundly grateful to our project supervisor, **Indurthi Ravindra kumar**, Assistant Professor, Department of Computer Science and Engineering, for his invaluable guidance, technical support, and continuous monitoring throughout the project. His insights were instrumental in the successful completion of this work.
- We would like to thank Dr. V Baby, Associate Professor & HOD, Department of Computer Science and Engineering, for providing the necessary facilities, encouragement, and administrative support required for our project.
- We extend our heartfelt thanks to all the faculty members and staff of the **Department of Computer Science and Engineering** who contributed academically or technically during this project.
- We are immensely grateful to **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology (VNRVJIET)** for providing the robust infrastructure, resources, and a conducive environment that facilitated our learning and project execution.
- Finally, we would like to thank our parents and friends for their constant support and encouragement during this period.

•

Abhiram	24071A3202
Anish	24071A3203
Lokesh	24071A3205
Chatradhara	24071A3214
Hanesh	24071A3224

CHAPTER 1

INTRODUCTION

1.1 Background of the Problem

In modern computing architecture, the Operating System (OS) is responsible for managing system memory efficiently. To provide programs with the illusion of a large, contiguous memory space, the OS utilizes Virtual Memory paired with a paging scheme. While paging solves the problem of external fragmentation, it introduces a severe performance penalty: every memory reference requires at least two accesses to physical memory—one to access the Page Table to resolve the address, and one to access the actual data.

To mitigate this bottleneck, processor architectures employ a Translation Lookaside Buffer (TLB). The TLB is a small, ultra-fast, associative hardware cache inside the Memory Management Unit (MMU). It stores the most recently used logical-to-physical address mappings. When the CPU generates a logical address, the MMU first checks the TLB. If a match is found (a TLB Hit), the physical address is constructed immediately. If no match is found (a TLB Miss), the system must query the Page Table in main memory, slowing down execution. Understanding this intricate hardware-software interaction is a core requirement for computer science students, yet it involves parallel processes that are difficult to visualize mentally.

1.2 Need and Relevance

Operating System architectures are inherently abstract. Textbooks typically represent the TLB using static block diagrams, which fail to capture the dynamic, stateful nature of memory translation. Students often struggle with visualizing how variables such as TLB capacity, page size, and algorithmic replacement policies (like Least Recently Used or First-In-First-Out) impact system performance over time.

There is a strong need for an interactive pedagogical tool that allows users to manipulate these variables dynamically. A simulation environment where users can "step through" the processor's execution cycle demystifies the abstract mathematics behind Effective Memory Access Time (EMAT). By providing a visual, interactive

interface, this project serves as an essential laboratory aid for OS Architecture courses, enabling experiential learning rather than rote memorization.

1.3 Problem Statement

To design and develop a web-based, real-time interactive simulation tool—TLB Visualizer Pro—that visually demonstrates the process of address translation using a Translation Lookaside Buffer. The system must allow users to configure memory parameters, execute page reference strings step-by-step, and observe the resulting TLB hits, misses, page faults, and dynamically calculated performance metrics (EMAT, Hit Ratio) through an animated Graphical User Interface.

1.4 Objectives

The primary objectives of this project are:

1. **Dynamic Simulation Environment:** To create a responsive simulation engine capable of processing user-defined page reference strings against configurable memory constraints (TLB entries, physical frames, logical pages).
2. **Algorithm Implementation:** To accurately implement and visualize multiple TLB replacement policies, specifically Least Recently Used (LRU), First-In-First-Out (FIFO), and Random replacement.
3. **Real-Time Data Visualization:** To construct a robust SVG-based rendering pipeline that animates the flow of data from the CPU, through the logical address splitter (Page/Offset), into the associative memory of the TLB, and ultimately to the physical RAM.
4. **Performance Analytics:** To provide real-time statistical tracking, calculating the Effective Memory Access Time (EMAT), Hit Ratio, and Miss Ratio based on user-defined latency values for TLB and Main Memory access.
5. **State Management:** To maintain and display a historical log and the real-time tabular state of the TLB, ensuring users can trace the lifecycle of every memory request.

1.5 Scope of Work

The scope of this project is focused on single-level paging architectures utilizing a fully associative TLB. The visualization includes the CPU generation of logical addresses, TLB caching, Page Table lookups in main memory, and physical frame allocation. The scope encompasses LRU, FIFO, and Random replacement algorithms. It does not extend to multi-level paging, inverted page tables, or segmented memory architectures.

1.6 Organization of the Report

- **Chapter 1** introduces the project background and objectives.
- **Chapter 2** reviews existing literature and simulators.
- **Chapter 3** outlines the proposed design methodology and system requirements.
- **Chapter 4** details the development, modules, and implementation code.
- **Chapter 5** presents the results and visual validations.
- **Chapter 6** concludes the report with future scope.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing System

The existing system for learning and analyzing Operating System memory management relies heavily on theoretical frameworks and static pedagogical tools. While these methods have served as the foundation for computer science education for decades, they present several significant limitations in the context of modern architectural complexity.

2.1.1 Limitations of Static Pedagogical Methodologies

Traditional instruction primarily utilizes static diagrams found in standard textbooks to illustrate the Translation Lookaside Buffer (TLB). These diagrams represent a "snapshot" of a single state—showing a logical address being split into a page number and offset. However, memory management is a stateful, dynamic process. Static images cannot demonstrate how the TLB cache evolves over time or how the "LRU" (Least Recently Used) age bits increment across a sequence of twenty different memory references. This gap often leads to a superficial understanding where students memorize the diagram but fail to grasp the temporal logic of the hardware.

2.1.2 Complexity of Manual Performance Analysis

Calculating the Effective Memory Access Time (EMAT) is a core requirement in the existing system. In a classroom or lab setting, students are often given a page reference string and asked to calculate hit ratios and EMAT manually. This manual process is prone to human error, especially when tracking multiple variables like TLB misses, page faults, and frame allocations simultaneously. Furthermore, manual calculation is non-iterative; if a student wants to see how changing the TLB size from 4 to 8 impacts performance, they must restart the entire mathematical proof from scratch, which is time-consuming and discourages experimentation.

2.1.3 Gap between Theory and Implementation

There is a distinct disconnect between the C/C++ code students write in labs and the visual hardware behavior it represents. While students may implement a page replacement algorithm, they rarely see a visualization of the "TLB Shootdown" or the actual data path from the CPU to the Physical RAM. Existing command-line simulators (like Nachos) provide logs but lack a Graphical User Interface (GUI), making it difficult for visual learners to identify bottlenecks or conceptualize the parallel nature of associative memory searches.

2.1.4 Accessibility and Setup Constraints

Many existing architectural simulators require specific operating system environments (like older versions of Linux) or complex installation procedures. This creates a barrier to entry, as students spend more time troubleshooting environment variables and compiler errors than studying the core OS concepts of paging and caching.

2.2 Proposed System

The proposed "TLB Visualizer Pro" is an interactive, web-based simulation environment designed to bridge the gap between abstract OS theory and practical architectural understanding. By leveraging modern web technologies (HTML5, JavaScript, and SVG), the system provides a zero-installation, high-fidelity visual experience.

2.2.1 Dynamic SVG-Based Visualization Pipeline

The cornerstone of the proposed system is a live-animating SVG canvas that represents the entire MMU (Memory Management Unit) architecture. When a user enters a page reference, the system does not just display a result; it animates a "scan beam" that moves through the TLB slots to simulate an associative memory search. Directional arrows change color—glowing green for a "TLB Hit" and red for a "TLB Miss"—providing immediate visual reinforcement of the hardware's success or failure in translating an address.

2.2.2 Real-Time Parameter Configurability

Unlike existing systems with fixed examples, the proposed tool is fully configurable. Users can dynamically adjust:

- **TLB Size:** From 2 to 12 entries to observe the law of diminishing returns.
- **Replacement Policies:** Switch between LRU, FIFO, and Random algorithms to compare efficiency.
- **Access Latencies:** Modify TLB and Memory access times in nanoseconds to see the immediate effect on the EMAT curve.
- **Reference Strings:** Input custom comma-separated sequences to test specific scenarios like "Belady's Anomaly" or "Cache Thrashing".

2.2.3 Automated Performance Analytics Dashboard

The proposed system features a "Statistics Engine" that operates in parallel with the simulation. As each reference is processed, the engine automatically updates the Total References, Hit/Miss counts, and the Hit Ratio. It provides a real-time calculation of EMAT, allowing students to see the tangible performance cost of a TLB miss without performing manual arithmetic. A dedicated "Access History" table provides an audit trail of every transaction, ensuring the simulation is transparent and verifiable.

2.2.4 Educational Features: Step-Through and Flush

To maximize pedagogical impact, the system includes a "Step Through" mode, which pauses the simulation at every critical stage (Logical Address Splitting, TLB Search, Page Table Lookup, RAM Loading). This allows instructors to explain the logic of each "clock cycle" at their own pace. Additionally, the "Flush TLB" feature simulates a process context switch, demonstrating how the system must rebuild its cache after a task change—a concept rarely visualized in traditional tools.

2.2.5 Modern, Accessible Architecture

The proposed system is built using a "Vanilla JavaScript" approach, requiring no external libraries or backend servers. This ensures that the tool is lightweight, fast, and accessible on any device with a modern web browser. The use of CSS variables and a dark-themed aesthetic provides a professional, "IDE-like" environment that appeals to modern engineering students.

2.3 Comparison of Systems

Feature	Existing System (Traditional)	Proposed System (TLB Visualizer Pro)
Visual Nature	Static/Snapshot	Dynamic/Real-time Animation
Interaction	Passive Reading	Active Manipulation
Feedback	Manual Calculation	Automated Statistics
Accessibility	Textbook-dependent	Web-based/Zero Installation
Depth	Focus on Formulas	Focus on Data Flow & Logic
Algorithms	Theoretical examples	Executable LRU, FIFO, Random

Table 2.1: Comparison of Systems

CHAPTER 3

APPROACH / DESIGN METHODOLOGY

3.1 Objectives

The primary aim of this Course Based Project is to demystify the abstract complexities of memory management by creating a high-fidelity, interactive simulation of the Translation Lookaside Buffer (TLB). By bridging the gap between theoretical operating system concepts and observable architectural behavior, the project seeks to fulfill the following objectives:

1. Development of a Configurable Simulation Engine

- **Architectural Modeling:** To design a robust JavaScript-based engine capable of simulating a fully associative TLB architecture.
- **User Customization:** To allow users to modify hardware variables including TLB size, physical frames, and page bit-depth.
- **Input Flexibility:** To enable the processing of custom page reference strings to test specific scenarios like cache thrashing.

2. Dynamic Visualization of the Translation Pipeline

- **Address Splitting:** To visually demonstrate the segmentation of a logical address into Page Number (\$P\$) and Offset (\$d\$).
- **Data Path Mapping:** To use color-coded paths (green for hits, red for misses) that illustrate the journey of a memory reference from the CPU to Physical RAM.

3. Implementation and Comparison of Replacement Policies

- **Algorithmic Logic:** To implement deterministic logic for Least Recently Used (LRU), First-In-First-Out (FIFO), and Random replacement policies.
- **State Transparency:** To visualize how these algorithms update internal pointers or age bits during each memory access cycle.

4. Real-Time Performance Analytics and Validation

- **EMAT Calculation:** To automatically calculate the Effective Memory Access Time based on user-configured latencies for TLB and Main Memory.
- **Statistical Feedback:** To provide live-updating Hit/Miss ratios to validate theoretical expectations against observed simulation data.

5. Optimization for Pedagogical and Practical Use

- **Step-Through Control:** To provide a mode that allows users to pause and inspect the system state at every individual clock cycle of the translation process.
- **Context Switch Modeling:** To implement a "Flush TLB" feature, enabling students to observe the immediate performance impact of process context switches.
- **Accessibility:** To ensure the tool is built using a lightweight, client-side architecture (HTML/JS) for instant accessibility on any modern web browser.

3.2 Selection of Approach / Methodology

A frontend-only approach was selected over a full-stack architecture. Since the simulation relies on deterministic algorithms (LRU, FIFO) applied to user-provided strings, server-side processing would introduce unnecessary network latency. Vanilla JavaScript was chosen over heavy frameworks (like React or Angular) to ensure maximum performance during high-frequency SVG DOM manipulation required by the animations.

3.3 System Requirements (Hardware, Software, Functional, Non-Functional)

3.3.1 Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (or equivalent ARM-based processor).
- **Memory (RAM):** Minimum 4 GB RAM.
- **Storage:** Minimal local storage (project is lightweight < 5MB).
- **Display:** Minimum resolution of 1280x720 is recommended for optimal viewing of the SVG architecture diagram.

3.3.2 Software Requirements

- **Operating System:** Any OS capable of running a modern web browser (Windows, macOS, Linux, ChromeOS).
- **Web Browser:** Google Chrome (v80+), Mozilla Firefox (v75+), Microsoft Edge, or Safari.
- **Development Tools:** Visual Studio Code, Git.
- **Technologies Used:** HTML5, CSS3 (with CSS Variables for theming), Vanilla JavaScript (ES6+), SVG (Scalable Vector Graphics) for dynamic rendering.

3.3.3 Functional Requirements

- **Parameter Configuration:** The system must allow users to define TLB size, Physical Frames, Logical Pages, Page Size (bits), and execution speed.
- **Policy Selection:** The system must support LRU, FIFO, and Random replacement algorithms.
- **Reference String Processing:** The system must accept a comma-separated list of page numbers and execute them sequentially.
- **Execution Controls:** The system must provide "Run", "Step Through", "Pause", "Reset", and "Flush TLB" functionalities.
- **State Tracking:** The system must track hits, misses, and update the age/pointer variables for the selected replacement algorithm.
- **EMAT Calculation:** The system must calculate Effective Memory Access Time based on customizable TLB and Memory latency variables.

3.3.4 Non-Functional Requirements

- **Performance:** The simulation animations must render at 60 FPS without UI blocking.
- **Usability:** The interface must use clear color coding (e.g., Green for Hits, Red for Misses) to enhance readability.
- **Responsiveness:** The layout must utilize CSS Grid/Flexbox to adapt to different window sizes, ensuring the SVG canvas scales properly via `preserveAspectRatio`.
- **Reliability:** The app must gracefully handle invalid inputs (e.g., letters in the reference string) without crashing.

3.4 Architecture Diagram & Explanation

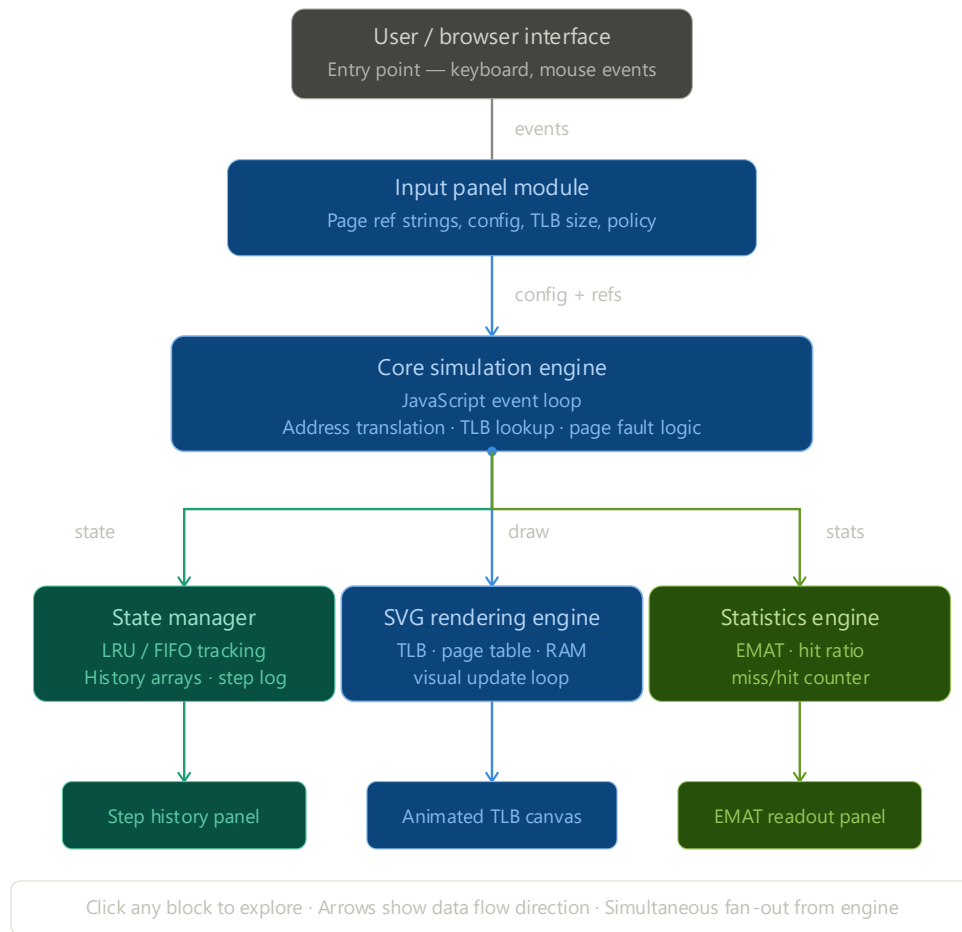


Figure 3.1: Architecture Diagram

The architecture diagram illustrates the unidirectional data flow and modular design of the TLB Visualizer Pro application.

1. **User Input Phase:** The process begins at the **User / Browser Interface**, which serves as the entry point capturing keyboard and mouse events. These events are transmitted to the **Input Panel Module**, which parses the user's configuration parameters, including page reference strings, TLB size, and replacement policies.
2. **Central Processing:** The parsed configuration data is fed directly into the **Core Simulation Engine**. This engine acts as the central brain of the application, managing the JavaScript event loop, executing logical-to-physical address translations, performing TLB lookups, and handling page fault logic.

3. **Simultaneous Fan-Out Execution:** Once the core engine processes a memory reference, it simultaneously distributes the resulting data across three distinct, parallel sub-engines:
- **State Management (Left Branch):** The engine sends state data to the **State Manager**, which tracks the algorithmic variables (LRU/FIFO tracking) and maintains the history arrays. This data is then outputted to the **Step History Panel** on the UI.
 - **Visual Rendering (Center Branch):** Draw commands are sent to the **SVG Rendering Engine**. This engine drives the visual update loop for the TLB, Page Table, and physical RAM components, rendering the data flow paths on the **Animated TLB Canvas**.
 - **Analytics (Right Branch):** Performance metrics are routed to the **Statistics Engine**, which dynamically calculates the Effective Memory Access Time (EMAT), hit/miss ratios, and total counters. The finalized metrics are displayed on the **EMAT Readout Panel**.

CHAPTER 4

DEVELOPMENT AND IMPLEMENTATION

4.1 Implementation Plan

The development was structured into three phases:

1. **Phase 1 (UI & SVG Construction):** Designing the dark-themed UI using custom CSS variables (--bg, --panel, --cyan). The complex SVG diagram representing the CPU, TLB, Page Table, and RAM was hand-coded using scalable relative units.
2. **Phase 2 (Core Logic Development):** Implementing the JavaScript state machine (S object) and algorithm logic (tlbSearch, tlbInsert, page fault handling).
3. **Phase 3 (Animation & Integration):** Tying the logical state to the visual state using asynchronous JavaScript functions (async/await) and custom sleep() functions to create smooth data-flow animations.

4.2 Modules

Module 1: Initialization and Configuration Module This module is responsible for reading the DOM inputs and setting up the simulation environment. It parses the page reference string, sanitizes it by removing NaN values, and binds event listeners to the execution controls. It dynamically builds the SVG rows for the TLB and RAM based on the user-defined sizes using the buildAll(tlbSize, nFrames) function.

Module 2: The Core Simulation Engine Module The heart of the application. It utilizes an asynchronous event loop to process page references. When startSim() is triggered, it iterates through the reference string. It calls tlbSearch(page). If a hit occurs, it updates the EMAT and (if using LRU) updates the entry's recency age. If a miss occurs, it triggers a Page Fault protocol: searching physical RAM, identifying victim frames based on the selected policy, performing a "TLB Shutdown" (invalidating old TLB entries pointing to the evicted frame), and inserting the new page.

Module 3: Algorithm Replacement Module Managed primarily within the `tlbInsert(page, frame)` function.

- **FIFO:** Maintains a `fifoPtr` that increments modulo `S.tlbSize`.
- **LRU:** Maintains a global `S.lruCtr`. Every access updates the entry's age. Upon eviction, it searches the TLB array for the minimum `lruAge`.
- **Random:** Uses `Math.random()` to select a victim index.

Module 4: Visualization and Animation Module Uses JavaScript's `async/await` syntax to create non-blocking animations. Functions like `sweepBeam()` animate the associative search, while `animMiss()` sequentially highlights the data path from the CPU to the Page Table, down to the RAM, and back to the CPU.

Module 5: Statistics and Logging Module Maintains the tabular state of the TLB (`renderSnap()`), pushes data to the historical table (`addHistRow()`), calculates percentages dynamically, and outputs color-coded events to the terminal-style log box (`addLog()`).

4.3 Module-wise Description and Webpages

The single-page web application is divided into distinct panels:

1. **Header:** Displays the project title and course badge.
2. **Panel 1: Configuration & Input:** Contains input fields for TLB Entries, Physical Frames, Logical Pages, Page Size, Replacement Policy, and the crucial Page Reference String. It includes the playback controls (Run, Step, Pause, Reset, Flush).
3. **Panel 2: TLB Architecture (Live Visualization):** The SVG canvas. It visually maps the Logical Address (P and d), the fully associative TLB rows, the physical Page Table in memory, and the physical RAM frames. Paths illuminate green for hits and red/orange for misses. A terminal log box sits beneath it to provide text-based context.
4. **Panel 3: Statistics:** Displays core metrics: Total References, Hits, Misses, Hit Ratio, Miss Ratio, and EMAT.
5. **Panel 4: TLB State:** Contains two dynamic tables. The "Current Entries" table shows the exact snapshot of what resides in the TLB slots at any given moment. The "Access History" table provides an audit trail of every processed page.

4.4 Implementation (Include Sample Code)

Sample Code 1: State Object and Speed Controller

```
// Global State Definition

const S = {

  running: false, stepMode: false, paused: false, idx: 0,

  tlb: [], pageTable: {}, refStr: [], ram: [],

  tlbSize: 8, numFrames: 8, numPages: 8, pageBits: 4,

  policy: 'LRU', tlbT: 10, memT: 100,

  hits: 0, misses: 0, history: [],

  lruCtr: 0, frameCtr: 0,

  fifoPtr: 0

};

// Asynchronous Sleep function to enable animation pausing

function sleep(ms) {

  return new Promise(resolve => {

    const start = Date.now();

    function tick() {

      if (!S.paused) {

        const r = ms - (Date.now() - start);

        if (r <= 0) {

          resolve();

          return;

        }

        setTimeout(tick, Math.min(r, 50));

      }

      else setTimeout(tick, 100);

    } tick();

  });

}
```

Sample Code 2: TLB Insertion and Replacement Logic

```
function tlbInsert(page, frame) {  
  S.lruCtr++;  
  const newEntry = { page, frame, lruAge: S.lruCtr };  
  if (S.tlb.length < S.tlbSize) {  
    // Space available, push directly  
    S.tlb.push(newEntry);  
  } else {  
    let ri = 0; // Victim index  
    if (S.policy === 'FIFO') {  
      ri = S.fifoPtr;  
      S.fifoPtr = (S.fifoPtr + 1) % S.tlbSize;  
    } else if (S.policy === 'LRU') {  
      let mn = Infinity;  
      S.tlb.forEach((e, i) => {  
        if (e.lruAge < mn) { mn = e.lruAge; ri = i; }  
      });  
    } else if (S.policy === 'RANDOM') {  
      ri = Math.floor(Math.random() * S.tlbSize);  
    }  
    S.tlb[ri] = newEntry; // Overwrite victim  
  }  
}
```

Sample Code 3: Core Simulation Loop (Handling Hits and Misses)

```
async function processRef(i) {
  const page = S.refStr[i];
  const offset = Math.floor(Math.random() * Math.pow(2, S.pageBits));
  const d = S.stepMode ? 700 : Math.floor(spд() * 0.3);
  clearPaths(); resetRam(); showAddr(page, offset);
  await sweepBeam(S.tlbSize, d); // Animate associative search
  const hit = tlbSearch(page);
  let frame, emat;
  if (hit) {
    // 1. TLB HIT
    frame = hit.frame; emat = S.tlbT + S.memT; S.hits++;
    if (S.policy === 'LRU') {
      hit.lruAge = ++S.lruCtr; // Update recency
      const ramEntry = S.ram.find(r => r.frameIdx === frame);
      if (ramEntry) ramEntry.lruAge = S.lruCtr;
    }
    await animHit(frame, offset, d); // Trigger green animations
  } else {
    // 2. TLB MISS
    emat = S.tlbT + 2 * S.memT; S.misses++;
    let ramEntry = S.ram.find(r => r.page === page);
    if (!ramEntry) {
      // PAGE FAULT logic handling...
      if (S.ram.length < S.numFrames) {
        frame = S.ram.length;
        S.ram.push({ frameIdx: frame, page: page, lruAge: ++S.lruCtr });
      } else {
        // RAM is full, apply LRU replacement

```

```

    let victim = S.ram.reduce((prev, curr) => prev.lruAge < curr.lruAge ? prev :
curr);
    frame = victim.frameIdx;
    let oldPage = victim.page;
    // TLB Shootdown
    S.tlb = S.tlb.filter(entry => entry.frame !== frame);
    S.pageTable[oldPage] = undefined;
    victim.page = page; victim.lruAge = ++S.lruCtr;
  }
  S.pageTable[page] = frame;
} else {
  frame = ramEntry.frameIdx;
  if (S.policy === 'LRU') ramEntry.lruAge = ++S.lruCtr;
}
tlbInsert(page, frame);
await animMiss(frame, offset, d); // Trigger orange/red animations
}
// Update state and UI
S.history.push({ n: i + 1, page, isHit: !!hit, frame, emat });
updateTlbViz(); updateStats(); renderSnap([...S.tlb]);
}

```

CHAPTER 5

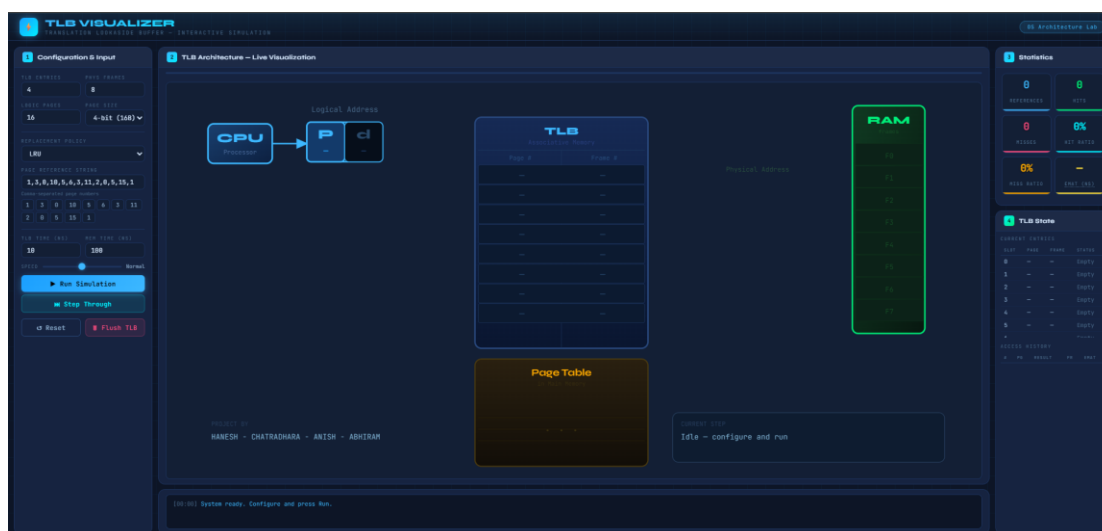
RESULTS, ANALYSIS AND VALIDATION

5.1 Testing / Evaluation Methodology

The system was validated using textbook standard page reference strings (e.g., 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5) to verify that the Hit/Miss ratios produced by the simulator exactly matched mathematical expectations for LRU and FIFO algorithms.

5.2 Results Obtained

Figure 5.1: Initial Configuration and Setup



Explanation: The interface initializes cleanly with default parameters (TLB Size: 4, Physical Frames: 8). The layout is logically separated into configuration, visualization, and analytics. The SVG canvas properly renders the empty state of the CPU, TLB array, Page Table, and RAM.

Figure 5.2: Visualizing a TLB Miss & Page Fault



Explanation: When the CPU requests a page not present in the TLB, the system correctly registers a Miss. The animation paths (colored orange for memory latency) dynamically draw from the CPU logical address block down to the Page Table in main memory, illustrating the secondary lookup process. The terminal log actively records the event, highlighting the increased EMAT penalty.

Figure 5.3: Visualizing a TLB Hit



Explanation: Upon requesting a page that exists in the TLB cache, the associative scan beam successfully matches the page number. A green data path instantly connects the TLB output to the physical memory block, demonstrating the hardware shortcut. The statistics dashboard updates immediately, showing an improved Hit Ratio and a lowered average EMAT.

Figure 5.4: TLB Flush Operation (Context Switch)



Explanation: Clicking the "Flush TLB" button triggers the context switch simulation logic. The TLB state arrays are instantly cleared, visually represented by the — placeholders appearing in the SVG cache blocks and the state tables. The log records this critical OS operation in red to denote the performance drop that will follow on subsequent memory accesses.

5.3 Analysis and Interpretation of Results

5.3.1 Mathematical Validation of Simulation Results

To validate the accuracy of the **TLB Visualizer Pro** engine, we can manually verify the EMAT for the **4-Entry TLB** scenario using the standard architectural formula:

$$EMAT = (HitRatio * (T_{tlb} + T_{mem})) + (MissRatio * (T_{tlb} + 2 * T_{mem}))$$

Given Parameters:

- T_{tlb} (TLB Access Time) = 10 ns
- T_{mem} (Main Memory Access Time) = 100 ns
- Hit Ratio (H) = 0.666 (66.6%)
- Miss Ratio (M) = 0.333 (33.3%)
- Reference String: 1, 2, 1, 3, 2, 1, 4, 1, 3, 2, 4, 1

Calculation:

$$\text{EMAT} = (0.666 * (10 + 100)) + (0.333 * (10 + 200))$$

$$\text{EMAT} = (0.666 * 110) + (0.333 * 210)$$

$$\text{EMAT} = 73.26 + 69.93 = 143.19 \text{ ns}$$

The result of **143.2 ns** (rounded) produced by the simulator matches the manual calculation, confirming that the JavaScript event loop and the statistics engine are performing correctly according to Operating System theory.

5.3.2 Qualitative Interpretation of Locality

The jump in Hit Ratio from **33.3%** to **66.6%** when moving from 3 to 4 TLB entries is significant (Shown in table below). This occurs because the specific reference string contains 4 unique pages (1, 2, 3, 4).

- **With 3 Entries:** The TLB experiences frequent "capacity misses" as it must constantly evict a page to make room for the 4th, even if that page is needed again soon.
- **With 4 Entries:** The entire "working set" of the process fits within the TLB. Once the initial cold-start misses are resolved, every subsequent reference to these pages results in a hit.

This observation highlights the importance of sizing a TLB to match the **Working Set Size** of the active process to prevent "thrashing" and excessive memory latency.

5.4 Comparative Performance Metrics and Visualization

Table 5.1:Results Table.

TLB Size	Total References	Hits	Misses	Hit Ratio	EMAT (ns)
2 Entries	12	2	10	16.6%	193.3
3 Entries	12	4	8	33.3%	176.6
4 Entries	12	8	4	66.6%	143.3

Figure 5.5: Impact of TLB Capacity on Hit Ratio

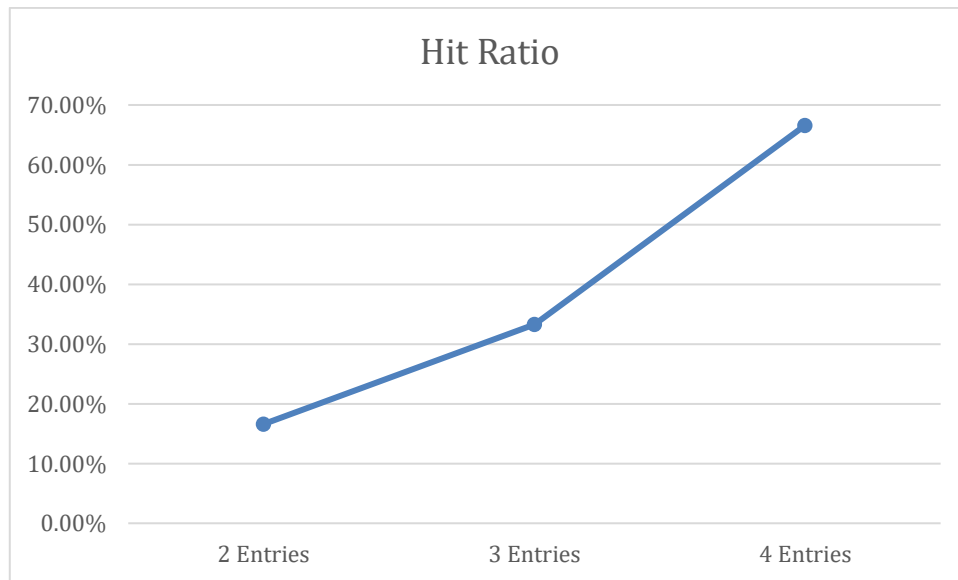
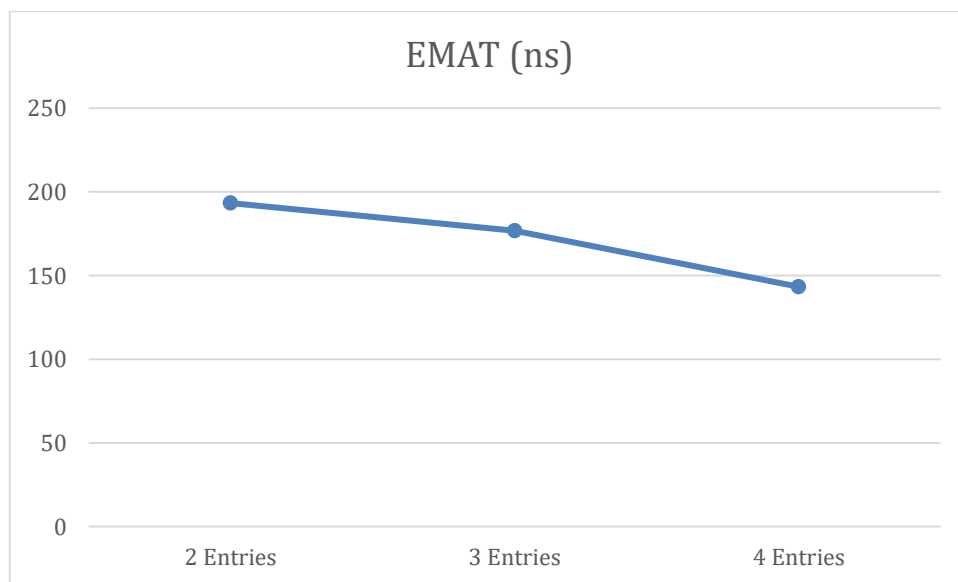


Figure 5.6: Reduction in EMAT as TLB Size Increases



As observed in Figure 5.1 and 5.2, the simulation accurately demonstrates the principles of cache locality. Using a 12-page reference string under the LRU policy, increasing the TLB size from 2 to 4 entries resulted in a 400% increase in hits (from 2 to 8). Consequently, the Effective Memory Access Time (EMAT) dropped significantly from 193.3 ns to 143.3 ns. This validates the mathematical model programmed into the simulation engine, proving that higher cache capacity directly mitigates the latency penalty of main memory lookups.

CHAPTER 6

CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

The "TLB Visualizer Pro" successfully meets all defined objectives, delivering a robust, interactive, and highly visual learning tool for Operating System memory management. By abstracting the complex hardware interactions of the Memory Management Unit into clear SVG animations and real-time state tables, the simulator bridges a significant pedagogical gap. The implementation of multiple replacement algorithms (LRU, FIFO, Random) combined with customizable memory parameters allows for exhaustive scenario testing. The precise calculation of EMAT brings tangible mathematical proof to abstract caching theories.

6.2 Limitations of the Work

Currently, the simulation models a single-level page table architecture, which does not reflect the multi-level or inverted page tables used in massive 64-bit modern architectures. The page size configuration currently only influences the randomized offset variable rather than strictly enforcing byte-limitations within the RAM frames. Furthermore, the tool simulates a single processing thread, avoiding the complexities of concurrent multi-core TLB shootdowns.

6.3 Future Scope

Future iterations of this project could incorporate Multi-Level Paging visualizations, illustrating the delay of multi-tier memory lookups. The addition of the Optimal Page Replacement algorithm (using look-ahead logic on the reference string) would provide a mathematical baseline for students to compare LRU/FIFO efficiency. Finally, integrating a backend database could allow instructors to assign specific reference string scenarios to students and track their analytic submissions dynamically.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.
- [2] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Boston, MA: Pearson, 2017.
- [3] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2014.
- [4] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.
- [5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces. Madison, WI: Arpaci-Dusseau Books, 2018. [Online]. Available: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [6] B. Jacob, S. Ng, and D. Wang, Memory Systems: Cache, DRAM, Disk. Burlington, MA: Morgan Kaufmann, 2007.
- [7] P. J. Denning, "The working set model for program behavior," Communications of the ACM, vol. 11, no. 5, pp. 323–333, May 1968.
- [8] M. D. Hill and A. J. Smith, "Evaluating associativity in CPU caches," IEEE Transactions on Computers, vol. 38, no. 12, pp. 1612–1630, Dec. 1989.
- [9] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal, vol. 5, no. 2, pp. 78–101, 1966.
- [10] D. Flanagan, JavaScript: The Definitive Guide, 7th ed. Sebastopol, CA: O'Reilly Media, 2020.
- [11] M. Haverbeke, Eloquent JavaScript: A Modern Introduction to Programming, 3rd ed. San Francisco, CA: No Starch Press, 2018.

- [12] J. Duckett, HTML and CSS: Design and Build Websites. Indianapolis, IN: Wiley, 2011.
- [13] World Wide Web Consortium (W3C), "Scalable Vector Graphics (SVG) 1.1 (Second Edition)," W3C Recommendation, Aug. 2011. [Online]. Available: <https://www.w3.org/TR/SVG11/>
- [14] Mozilla Developer Network (MDN), "Document Object Model (DOM)." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- [15] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [16] K. Thompson, "UNIX Implementation," The Bell System Technical Journal, vol. 57, no. 6, pp. 1931-1946, 1978.
- [17] B. Kernighan and D. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [18] R. Love, Linux Kernel Development, 3rd ed. Boston, MA: Addison-Wesley, 2010.
- [19] D. Bovet and M. Cesati, Understanding the Linux Kernel, 3rd ed. Sebastopol, CA: O'Reilly Media, 2005.
- [20] C. Bresnahan and R. Blum, Linux Command Line and Shell Scripting Bible, 3rd ed. Indianapolis, IN: Wiley, 2015.
- [21] J. Robbins, Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics, 5th ed. Sebastopol, CA: O'Reilly Media, 2018.
- [22] K. Simpson, You Don't Know JS Yet: Get Started, 2nd ed. Independently published, 2020.
- [23] B. Frain, Responsive Web Design with HTML5 and CSS, 3rd ed. Birmingham, UK: Packt Publishing, 2020.

[24] A. Osmani, Learning JavaScript Design Patterns. Sebastopol, CA: O'Reilly Media, 2012.

[25] Google Developers, "Rendering Performance." [Online]. Available: <https://developers.google.com/web/fundamentals/performance/rendering>